Event Bus

Purpose

The Event Bus is the communication backbone of Shio Runtime.

While the Character State represents the current observable condition of the character, the Event Bus represents everything that happens over time.

Character State answers the question:

> **"What is true right now?"**

The Event Bus answers the question:

> **"What just happened?"**

Both concepts are fundamental.

Neither replaces the other.

Character State vs Events

Understanding the difference between state and events is essential.

Character State is persistent.

Events are transient.

For example:

```text id="evt01" Character State

Mode = Speaking Emotion = Happy Activity = Replying
```

```
describes the current situation.

Whereas:

```text id="evt02"
SpeechStarted

SentenceFinished

ToolCompleted
```
```

```
AnimationEnded

EmotionChanged
```

describe things that happened.

An event is consumed.

A state is observed.

Why Both Are Needed

Using only Character State would lose important information.

For example:

```text id="evt03" SentenceFinished

```
cannot be represented as state.

Likewise:

```text id="evt04"
Mode = Speaking
```

should not be emitted continuously as events.

The Runtime therefore distinguishes between:

- long-lived state;
- short-lived events.

Event Flow

Events propagate through the Runtime.

They are never targeted toward a specific Runtime.

Instead they are published.

Interested Runtime systems decide whether they should react.

```text id="evt05" Speech Runtime

    |

SentenceGenerated

    |

——————————► Event Bus ◄——————————

    |

    ├——————► Experience Layer

    ├——————► Avatar Runtime

    ├——————► Debug Runtime

    ├——————► Overlay

    └——————► Future Runtime
```

``` The publisher does not know who will consume the event. Consumers do not know who produced it. This preserves Runtime independence.

---

# Event Categories

Events fall into several categories.

## Speech Events

Examples: - ListeningStarted - ListeningStopped - RecognitionCompleted - SentenceGenerated - SpeechStarted - SpeechFinished

---

## Character Events

Examples: - ModeChanged - EmotionChanged - AttentionChanged - ActivityChanged These events are usually emitted after the Character State changes.

---
```

Avatar Events

Examples: - AnimationStarted - AnimationFinished - GestureStarted - GestureFinished - BlinkCompleted

---

Action Events

Examples: - ToolRequested - ToolStarted - ToolCompleted - ToolFailed - PermissionDenied

---

Experience Events

Examples: - ReactionStarted - ReactionCompleted - JokeSelected - SilenceSelected These events belong exclusively to the Experience Layer.

System Events

Examples: - RuntimeInitialized - RuntimePaused - RuntimeResumed - RuntimeShutdown These events describe Runtime lifecycle rather than character behavior.

Architectural Rules

The Event Bus follows several mandatory rules.

Events are immutable.

Once published, an event never changes.

Events describe the past.

Events never describe intentions. Correct: `text id="evt06" SpeechFinished`

Incorrect:

```text id="evt07" SpeechShouldFinish

```
Intentions belong elsewhere.

Events describe completed facts.

```
```

```
## Events never replace Character State.

An event may update Character State.

It never becomes Character State.

---

## Runtime systems publish.

Runtime systems subscribe.

Runtime systems never directly invoke event handlers belonging to another
Runtime.

The Event Bus performs delivery.

---

# Event Ordering

Whenever both Character State and Events are involved, the following order
should be respected:

```text id="evt08"
Action

↓

Character State Updated

↓

Event Published

↓

Runtime Reactions
```

This guarantees that every Runtime observing the event sees the newest Character State.

Future Evolution

The Event Bus is expected to become the central communication mechanism for every future Runtime.

Potential future publishers include:

- Memory Layer;
```

- Scheduler Runtime;
- Streaming Runtime;
- External Service Adapters;
- Plugin Runtime.

The addition of new Runtime systems should require only new subscriptions rather than changes to existing publishers.

This property allows Shio Runtime to scale without increasing coupling between independent Runtime systems.